

Priority Queues: Binary Heaps

Data Structures

Data Structures and Algorithms

This lesson

In-place heap sort algorithm. For this, we will first turn a given array into a heap by permuting its elements.

Turn Array into a Heap

BuildHeap($A[1 \dots n]$)

$size \leftarrow n$

for i from $\lfloor n/2 \rfloor$ downto 1:

 SiftDown(i)

BuildHeap($A[1 \dots n]$)

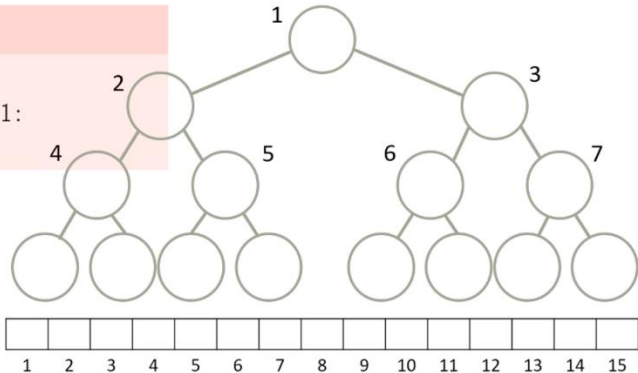
$size \leftarrow n$

for i from $\lfloor n/2 \rfloor$ downto 1:

 SiftDown(i)

Heap:

size = 15



BuildHeap($A[1 \dots n]$)

$size \leftarrow n$

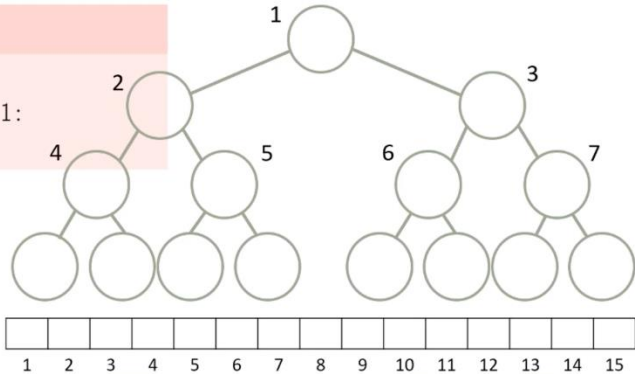
for i from $\lfloor n/2 \rfloor$ downto 1:

 SiftDown(i)

Heap:

size = 15

nodes = 15, sub-trees = 15



BuildHeap($A[1 \dots n]$)

$size \leftarrow n$

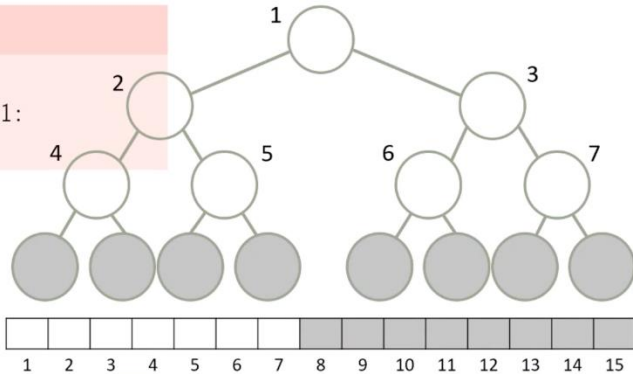
for i from $\lfloor n/2 \rfloor$ downto 1:

 SiftDown(i)

Heap:

size = 15

nodes = 15, sub-trees = 15



BuildHeap($A[1 \dots n]$)

$size \leftarrow n$

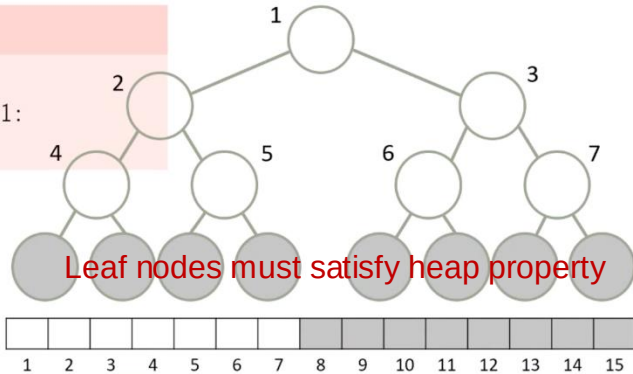
for i from $\lfloor n/2 \rfloor$ downto 1:

 SiftDown(i)

Heap:

size = 15

nodes = 15, sub-trees = 15



BuildHeap($A[1 \dots n]$)

$size \leftarrow n$

for i from $\lfloor n/2 \rfloor$ downto 1:

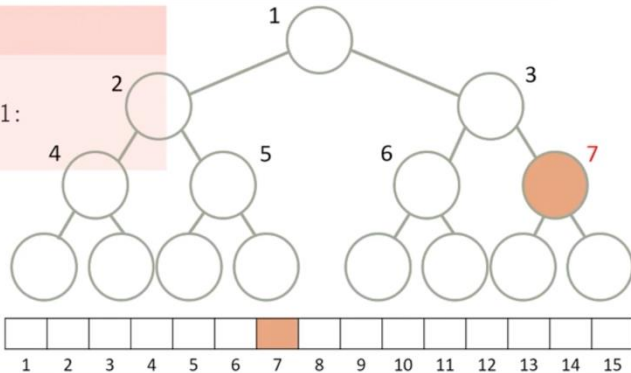
 SiftDown(i)

Heap:

size = 15

nodes = 15, sub-trees = 15

First potential node violation



BuildHeap($A[1 \dots n]$)

$size \leftarrow n$

for i from $\lfloor n/2 \rfloor$ downto 1:

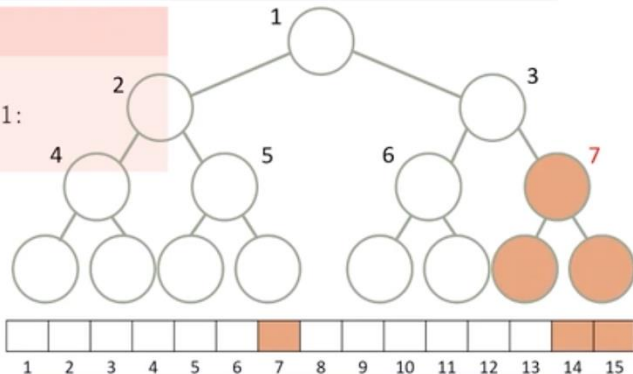
 SiftDown(i)

Heap:

size = 15

nodes = 15, sub-trees = 15

First potential tree violation



BuildHeap($A[1 \dots n]$)

$size \leftarrow n$

for i from $\lfloor n/2 \rfloor$ downto 1:

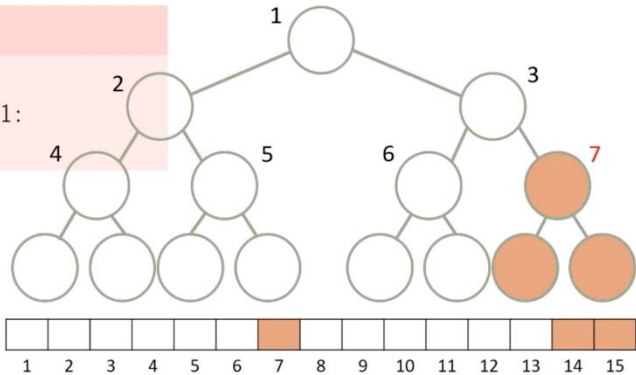
 SiftDown(i)

Heap:

size = 15

nodes = 15, sub-trees = 15

Call *SiftDown* for node 7

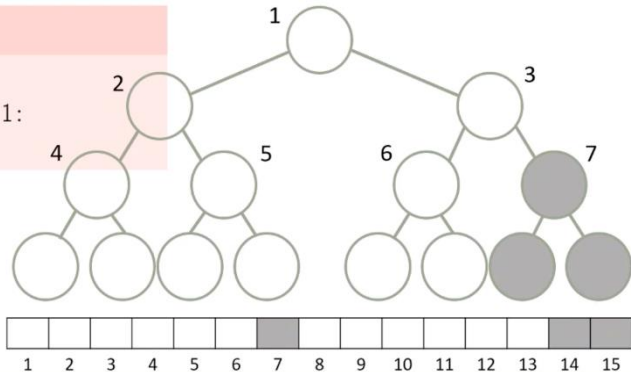


BuildHeap($A[1 \dots n]$)

$size \leftarrow n$
for i from $\lfloor n/2 \rfloor$ downto 1:
 SiftDown(i)

Heap:
 $size = 15$
 $nodes = 15$, sub-trees = 15

Heap Property **satisfied**



BuildHeap($A[1 \dots n]$)

$size \leftarrow n$

for i from $\lfloor n/2 \rfloor$ downto 1:

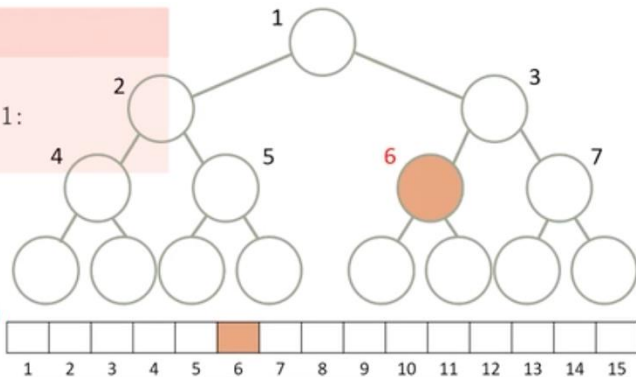
 SiftDown(i)

Heap:

size = 15

nodes = 15, sub-trees = 15

Next potential node violation



BuildHeap($A[1 \dots n]$)

$size \leftarrow n$

for i from $\lfloor n/2 \rfloor$ downto 1:

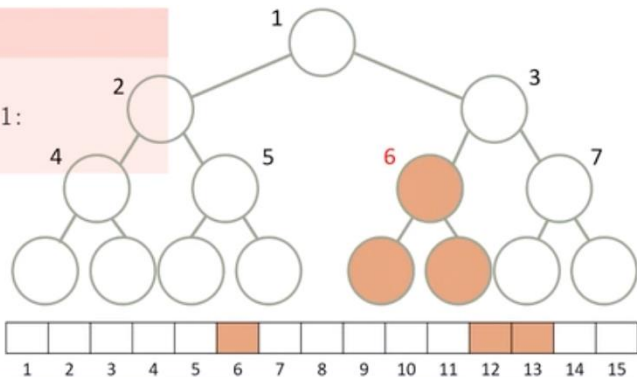
 SiftDown(i)

Heap:

size = 15

nodes = 15, sub-trees = 15

Next potential tree violation



BuildHeap($A[1 \dots n]$)

$size \leftarrow n$

for i from $\lfloor n/2 \rfloor$ downto 1:

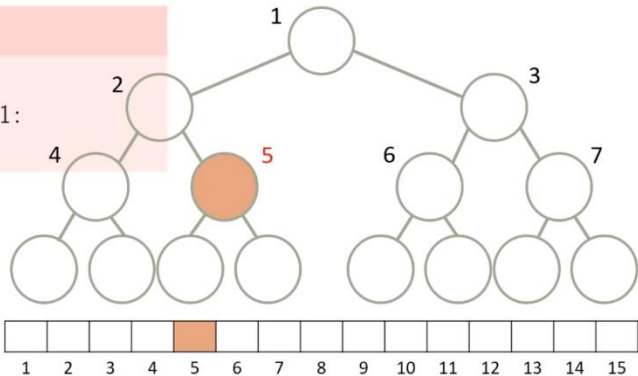
 SiftDown(i)

Heap:

size = 15

nodes = 15, sub-trees = 15

Next potential node violation



BuildHeap($A[1 \dots n]$)

$size \leftarrow n$

for i from $\lfloor n/2 \rfloor$ downto 1:

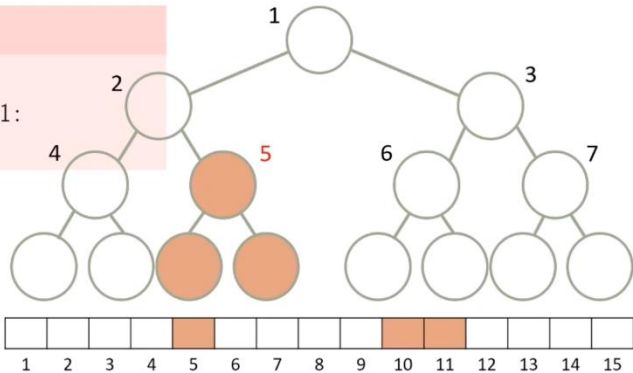
 SiftDown(i)

Heap:

size = 15

nodes = 15, sub-trees = 15

Next potential tree violation

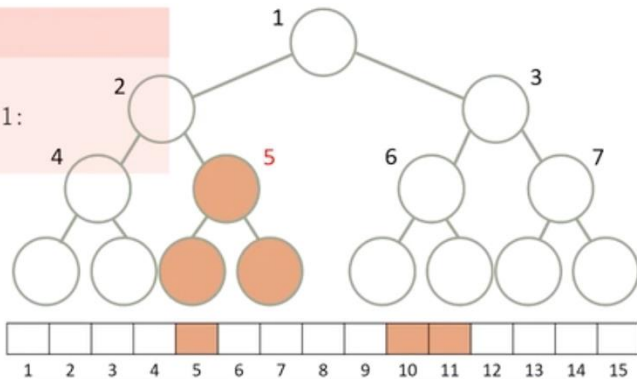


BuildHeap($A[1 \dots n]$)

```
size  $\leftarrow n$   
for  $i$  from  $\lfloor n/2 \rfloor$  downto 1:  
    SiftDown( $i$ )
```

Heap:
size = 15
nodes = 15, sub-trees = 15

Call *SiftDown* for node 5



BuildHeap($A[1 \dots n]$)

$size \leftarrow n$

for i from $\lfloor n/2 \rfloor$ downto 1:

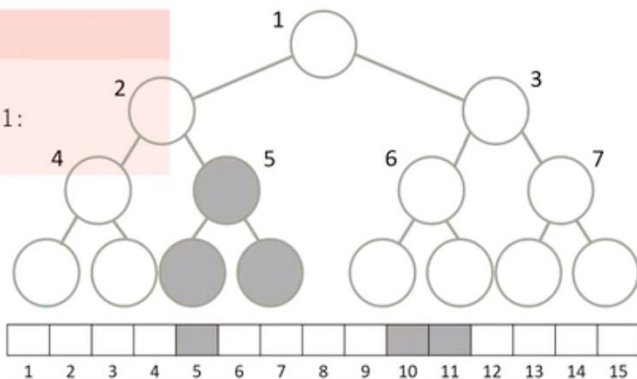
 SiftDown(i)

Heap:

size = 15

nodes = 15, sub-trees = 15

Heap Property **satisfied**



BuildHeap($A[1 \dots n]$)

$size \leftarrow n$

for i from $\lfloor n/2 \rfloor$ downto 1:

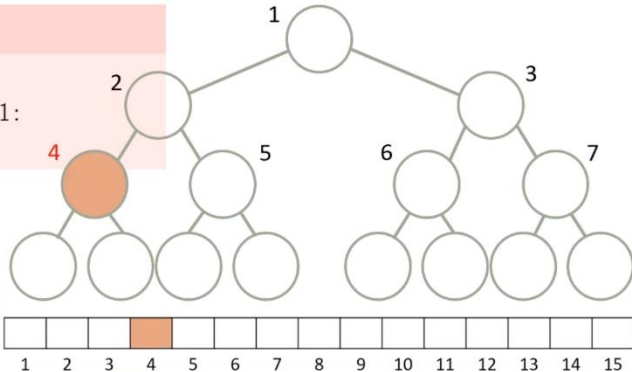
 SiftDown(i)

Heap:

size = 15

nodes = 15, sub-trees = 15

Next potential node violation



BuildHeap($A[1 \dots n]$)

$size \leftarrow n$

for i from $\lfloor n/2 \rfloor$ downto 1:

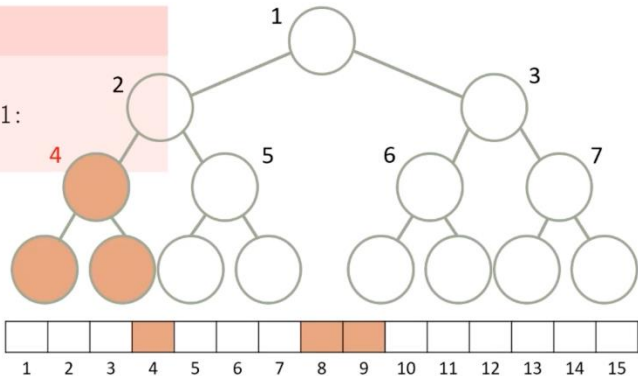
 SiftDown(i)

Heap:

size = 15

nodes = 15, sub-trees = 15

Next potential tree violation



BuildHeap($A[1 \dots n]$)

$size \leftarrow n$

for i from $\lfloor n/2 \rfloor$ downto 1:

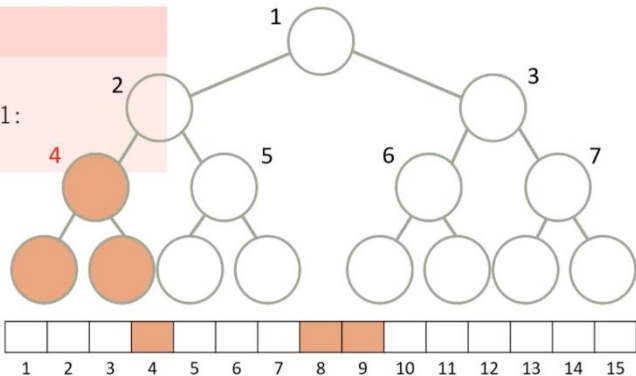
 SiftDown(i)

Heap:

size = 15

nodes = 15, sub-trees = 15

Call *SiftDown* for node 4



BuildHeap($A[1 \dots n]$)

$size \leftarrow n$

for i from $\lfloor n/2 \rfloor$ downto 1:

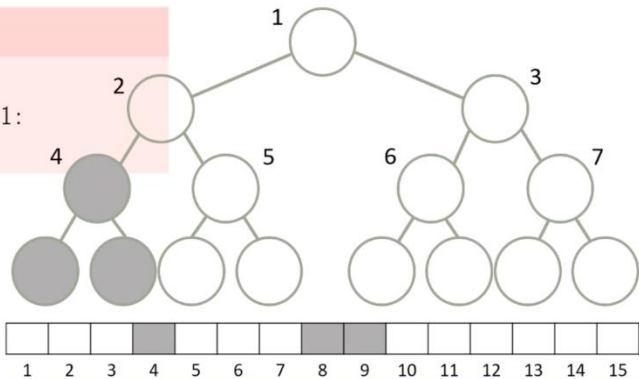
 SiftDown(i)

Heap:

size = 15

nodes = 15, sub-trees = 15

Heap Property **satisfied**



BuildHeap($A[1 \dots n]$)

$size \leftarrow n$

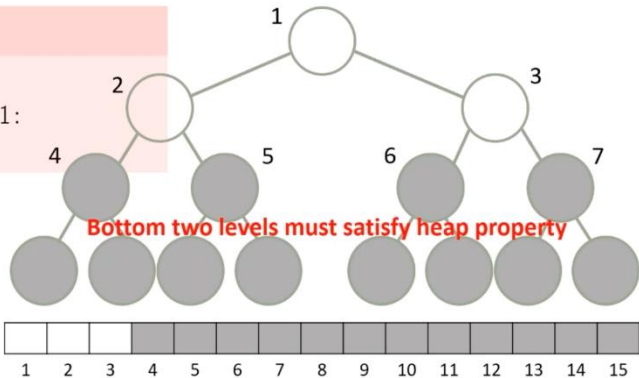
for i from $\lfloor n/2 \rfloor$ downto 1:

 SiftDown(i)

Heap:

size = 15

nodes = 15, sub-trees = 15



BuildHeap($A[1 \dots n]$)

$$size \leftarrow n$$

```
for  $i$  from  $\lfloor n/2 \rfloor$  downto 1:
```

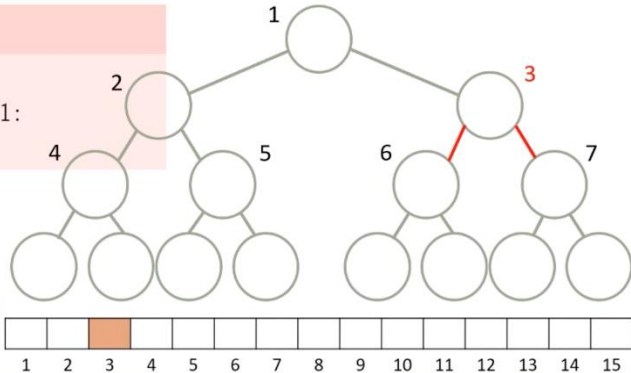
$$\text{SiftDown}(i)$$

Heap:

size = 15

nodes = 15, sub-trees = 15

Next potential node violation



BuildHeap($A[1 \dots n]$)

$size \leftarrow n$

for i from $\lfloor n/2 \rfloor$ downto 1:

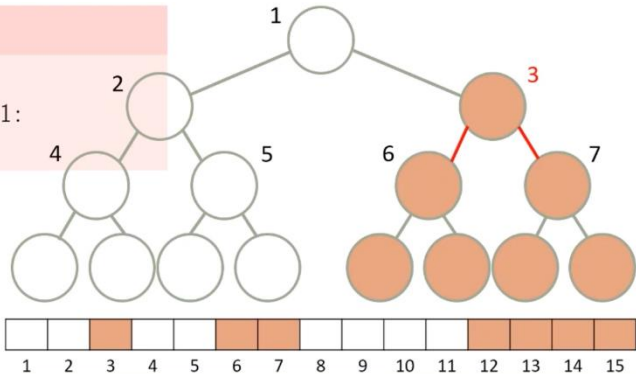
 SiftDown(i)

Heap:

size = 15

nodes = 15, sub-trees = 15

Call *SiftDown* for node 3



BuildHeap($A[1 \dots n]$)

$size \leftarrow n$

for i from $\lfloor n/2 \rfloor$ downto 1:

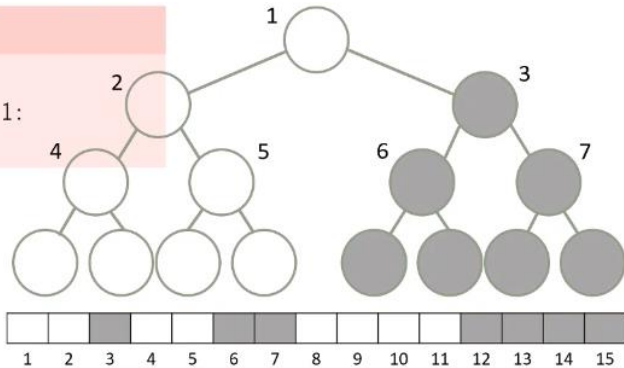
 SiftDown(i)

Heap:

$size = 15$

nodes = 15, sub-trees = 15

Heap Property **satisfied**



BuildHeap($A[1 \dots n]$)

$size \leftarrow n$

for i from $\lfloor n/2 \rfloor$ downto 1:

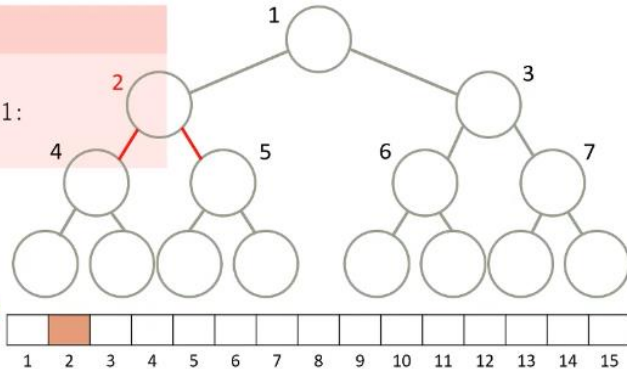
 SiftDown(i)

Heap:

size = 15

nodes = 15, sub-trees = 15

Next potential node violation



BuildHeap($A[1 \dots n]$)

$size \leftarrow n$

for i from $\lfloor n/2 \rfloor$ downto 1:

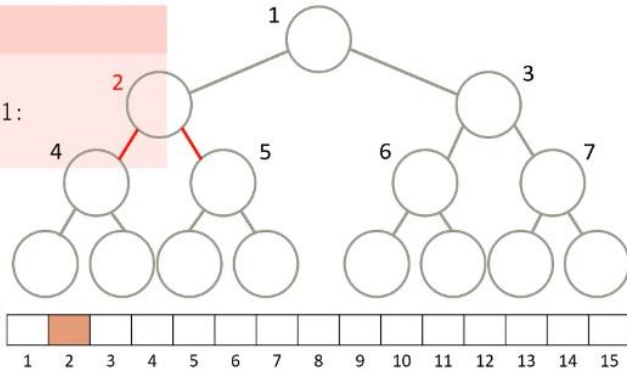
 SiftDown(i)

Heap:

size = 15

nodes = 15, sub-trees = 15

Next potential node violation

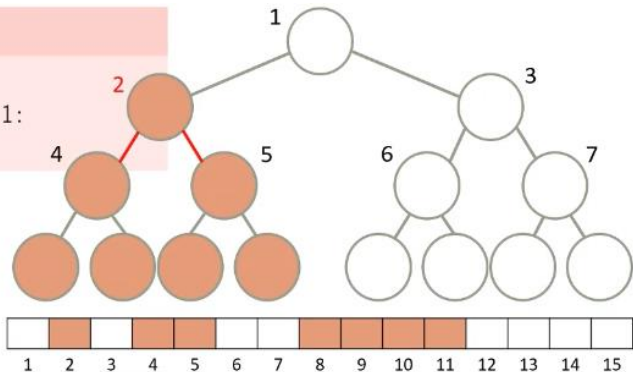


BuildHeap($A[1 \dots n]$)

```
size  $\leftarrow n$   
for  $i$  from  $\lfloor n/2 \rfloor$  downto 1:  
    SiftDown( $i$ )
```

Heap:
size = 15
nodes = 15, sub-trees = 15

Call *SiftDown* for node 2



BuildHeap($A[1 \dots n]$)

$size \leftarrow n$

for i from $\lfloor n/2 \rfloor$ downto 1:

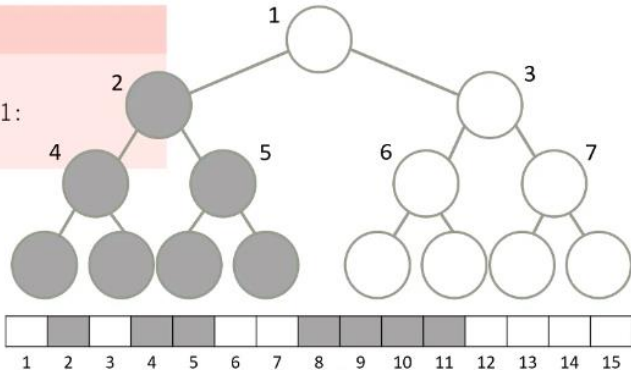
 SiftDown(i)

Heap:

size = 15

nodes = 15, sub-trees = 15

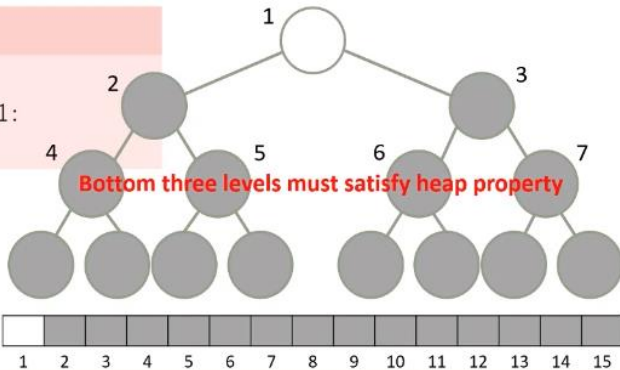
Heap Property **satisfied**



BuildHeap($A[1 \dots n]$)

$size \leftarrow n$
for i from $\lfloor n/2 \rfloor$ downto 1:
 SiftDown(i)

Heap:
size = 15
nodes = 15, sub-trees = 15



BuildHeap($A[1 \dots n]$)

$size \leftarrow n$

for i from $\lfloor n/2 \rfloor$ downto 1:

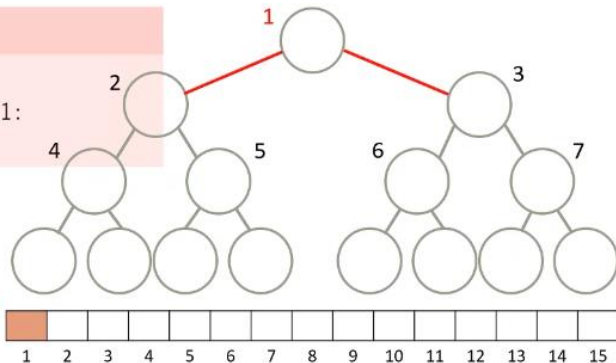
 SiftDown(i)

Heap:

size = 15

nodes = 15, sub-trees = 15

Only potential node violation



BuildHeap($A[1 \dots n]$)

$size \leftarrow n$

for i from $\lfloor n/2 \rfloor$ downto 1:

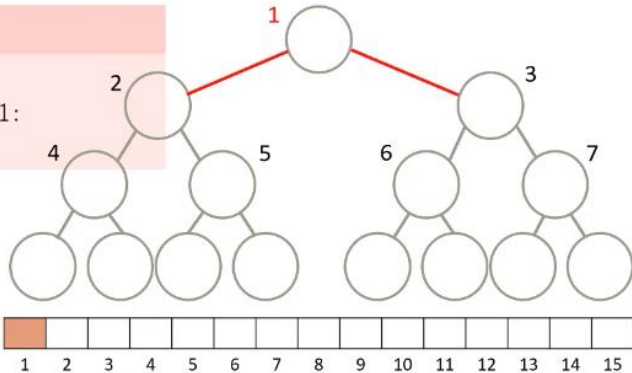
 SiftDown(i)

Heap:

size = 15

nodes = 15, sub-trees = 15

Call *SiftDown* for root



BuildHeap($A[1 \dots n]$)

$size \leftarrow n$

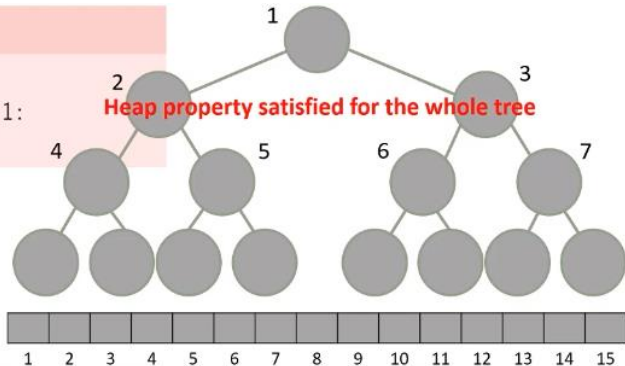
for i from $\lfloor n/2 \rfloor$ downto 1:

 SiftDown(i)

Heap:

size = 15

nodes = 15, sub-trees = 15



- We repair the heap property going from bottom to top.

- We repair the heap property going from bottom to top.
- Initially, the heap property is satisfied in all the leaves (i.e., subtrees of depth 0).

- We repair the heap property going from bottom to top.
- Initially, the heap property is satisfied in all the leaves (i.e., subtrees of depth 0).
- We then start repairing the heap property in all subtrees of depth 1.

- We repair the heap property going from bottom to top.
- Initially, the heap property is satisfied in all the leaves (i.e., subtrees of depth 0).
- We then start repairing the heap property in all subtrees of depth 1.
- When we reach the root, the heap property is satisfied in the whole tree.

- We repair the heap property going from bottom to top.
- Initially, the heap property is satisfied in all the leaves (i.e., subtrees of depth 0).
- We then start repairing the heap property in all subtrees of depth 1.
- When we reach the root, the heap property is satisfied in the whole tree.
- [Online visualization](#)

- We repair the heap property going from bottom to top.
- Initially, the heap property is satisfied in all the leaves (i.e., subtrees of depth 0).
- We then start repairing the heap property in all subtrees of depth 1.
- When we reach the root, the heap property is satisfied in the whole tree.
- [Online visualization](#)
- Running time: $O(n \log n)$

In-place Heap Sort

HeapSort($A[1 \dots n]$)

BuildHeap(A) { $size = n$ }

repeat $(n - 1)$ times:

 swap $A[1]$ and $A[size]$

$size \leftarrow size - 1$

 SiftDown(1)

	1	2	3	4	5	6	7
A	17	18	35	14	12	29	7

In-place Heap Sort

HeapSort($A[1 \dots n]$)

BuildHeap(A) { $size = n$ }

repeat $(n - 1)$ times:

 swap $A[1]$ and $A[size]$

$size \leftarrow size - 1$

 SiftDown(1)

	1	2	3	4	5	6	7
A	17	18	35	14	12	29	7

In-place Heap Sort

HeapSort($A[1 \dots n]$)

BuildHeap(A) { $size = n$ }

repeat $(n - 1)$ times:

 swap $A[1]$ and $A[size]$

$size \leftarrow size - 1$

 SiftDown(1)

	1	2	3	4	5	6	7
A	17	18	35	14	12	29	7

In-place Heap Sort

HeapSort($A[1 \dots n]$)

BuildHeap(A) { $size = n$ }

repeat $(n - 1)$ times:

 swap $A[1]$ and $A[size]$

$size \leftarrow size - 1$

 SiftDown(1)

	1	2	3	4	5	6	7
A	17	18	35	14	12	29	7

In-place Heap Sort

HeapSort($A[1 \dots n]$)

BuildHeap(A) { $size = n$ }

repeat $(n - 1)$ times:

 swap $A[1]$ and $A[size]$

$size \leftarrow size - 1$

 SiftDown(1)

	1	2	3	4	5	6	7
A	17	18	35	14	12	29	7

In-place Heap Sort

HeapSort($A[1 \dots n]$)

BuildHeap(A) { $size = n$ }

repeat $(n - 1)$ times:

 swap $A[1]$ and $A[size]$

$size \leftarrow size - 1$

 SiftDown(1)

	1	2	3	4	5	6	7
A	35	18	17	14	12	29	7

In-place Heap Sort

HeapSort($A[1 \dots n]$)

BuildHeap(A) { $size = n$ }

repeat $(n - 1)$ times:

 swap $A[1]$ and $A[size]$

$size \leftarrow size - 1$

 SiftDown(1)

	1	2	3	4	5	6	7
A	35	18	29	14	12	17	7

In-place Heap Sort

HeapSort($A[1 \dots n]$)

BuildHeap(A) { $size = n$ }

repeat $(n - 1)$ times:

 swap $A[1]$ and $A[size]$

$size \leftarrow size - 1$

 SiftDown(1)

	1	2	3	4	5	6	7
A	7	18	29	14	12	17	35

In-place Heap Sort

HeapSort($A[1 \dots n]$)

BuildHeap(A) { $size = n$ }

repeat $(n - 1)$ times:

 swap $A[1]$ and $A[size]$

$size \leftarrow size - 1$

 SiftDown(1)

	1	2	3	4	5	6	7
A	29	18	7	14	12	17	35

In-place Heap Sort

HeapSort($A[1 \dots n]$)

BuildHeap(A) { $size = n$ }

repeat $(n - 1)$ times:

 swap $A[1]$ and $A[size]$

$size \leftarrow size - 1$

 SiftDown(1)

	1	2	3	4	5	6	7
A	29	18	17	14	12	7	35

In-place Heap Sort

HeapSort($A[1 \dots n]$)

BuildHeap(A) { $size = n$ }

repeat $(n - 1)$ times:

 swap $A[1]$ and $A[size]$

$size \leftarrow size - 1$

 SiftDown(1)

	1	2	3	4	5	6	7
A	7	18	17	14	12	29	35

In-place Heap Sort

HeapSort($A[1 \dots n]$)

BuildHeap(A) { $size = n$ }

repeat $(n - 1)$ times:

 swap $A[1]$ and $A[size]$

$size \leftarrow size - 1$

 SiftDown(1)

	1	2	3	4	5	6	7
A	18	7	17	14	12	29	35

In-place Heap Sort

HeapSort($A[1 \dots n]$)

BuildHeap(A) { $size = n$ }

repeat $(n - 1)$ times:

 swap $A[1]$ and $A[size]$

$size \leftarrow size - 1$

 SiftDown(1)

	1	2	3	4	5	6	7
A	18	14	17	7	12	29	35

In-place Heap Sort

HeapSort($A[1 \dots n]$)

BuildHeap(A) { $size = n$ }

repeat $(n - 1)$ times:

 swap $A[1]$ and $A[size]$

$size \leftarrow size - 1$

 SiftDown(1)

	1	2	3	4	5	6	7
A	12	14	17	7	18	29	35

In-place Heap Sort

HeapSort($A[1 \dots n]$)

BuildHeap(A) { $size = n$ }

repeat $(n - 1)$ times:

 swap $A[1]$ and $A[size]$

$size \leftarrow size - 1$

 SiftDown(1)

	1	2	3	4	5	6	7
A	17	14	12	7	18	29	35

In-place Heap Sort

HeapSort($A[1 \dots n]$)

BuildHeap(A) { $size = n$ }

repeat $(n - 1)$ times:

 swap $A[1]$ and $A[size]$

$size \leftarrow size - 1$

 SiftDown(1)

	1	2	3	4	5	6	7
A	7	14	12	17	18	29	35

In-place Heap Sort

HeapSort($A[1 \dots n]$)

BuildHeap(A) { $size = n$ }

repeat $(n - 1)$ times:

 swap $A[1]$ and $A[size]$

$size \leftarrow size - 1$

 SiftDown(1)

	1	2	3	4	5	6	7
A	14	7	12	17	18	29	35

In-place Heap Sort

HeapSort($A[1 \dots n]$)

BuildHeap(A) { $size = n$ }

repeat $(n - 1)$ times:

 swap $A[1]$ and $A[size]$

$size \leftarrow size - 1$

 SiftDown(1)

	1	2	3	4	5	6	7
A	7	12	14	17	18	29	35

In-place Heap Sort

HeapSort($A[1 \dots n]$)

BuildHeap(A) { $size = n$ }

repeat $(n - 1)$ times:

 swap $A[1]$ and $A[size]$

$size \leftarrow size - 1$

 SiftDown(1)

	1	2	3	4	5	6	7
A	7	12	14	17	18	29	35

Building Running Time

- The running time of `BuildHeap` is $O(n \log n)$ since we call `SiftDown` for $O(n)$ nodes.

Building Running Time

- The running time of `BuildHeap` is $O(n \log n)$ since we call `SiftDown` for $O(n)$ nodes.
- If a node is already close to the leaves, then sifting it down is fast.

Building Running Time

- The running time of `BuildHeap` is $O(n \log n)$ since we call `SiftDown` for $O(n)$ nodes.
- If a node is already close to the leaves, then sifting it down is fast.
- We have many such nodes!

Building Running Time

- The running time of `BuildHeap` is $O(n \log n)$ since we call `SiftDown` for $O(n)$ nodes.
- If a node is already close to the leaves, then sifting it down is fast.
- We have many such nodes!
- Was our estimate of the running time of `BuildHeap` too pessimistic?

Question

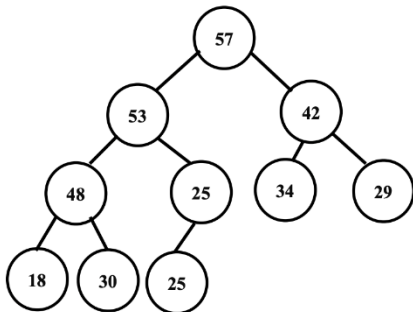
The array A below represents a binary heap implementation of a priority queue of items. Only the integer key values of the items are shown. The priority queue is arranged with the lowest-value key first.

	0	1	2	3	4	5	6	7	8	9
A		3	14	91	76	21	99	110	87	77

- (a) Draw the binary min-heap structure (tree) represented by A.
- (b) Draw the heap structure after the lowest-key item has been removed.
- (c) Draw the heap structure after a new item with key **1** has been added.

Question

(a) Insert 55 in the max-heap below. Show all the changes to the heap.



(b) Show the array that stores the above heap. Start at index 1.

(c) Rewrite the array (for the above heap) after each change it goes through during the insert 55 operation.

Question

Given the pseudocode below, use the HeapSort algorithm to sort the array $A=[22,13,17,11,10,14,12,15]$. Show each step of the process, including the building of the heap and each swap operation, until the array is sorted in ascending order.

HeapSort($A[1\dots n]$)

 BuildHeap(A)

 repeat $(n - 1)$ times:
 swap $A[1]$ and $A[\text{size}]$
 $\text{size} \leftarrow \text{size} - 1$
 SiftDown(1)

BuildHeap($A[1\dots n]$)

$\text{size} \leftarrow n$
 for i from $\lfloor n/2 \rfloor$ downto 1:
 SiftDown(i)